

Le Hachage

Géraldine Del Mondo

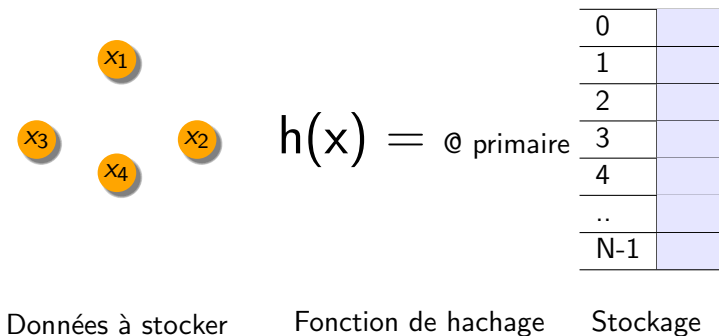
Département Architecture des Systèmes d'Information

Institut National des Sciences Appliquées - Rouen

geraldine.del_mondo@insa-rouen.fr

Le Hachage : principe (1)

→ Permettre l'amélioration des performances d'accès aux données



Le Hachage : principe (2)

$$h(x_4) = h(x_2)$$

Collision

0	x_1
1	
2	
3	x_2
4	x_3
..	
N-1	

x_4 va être placé à un autre emplacement : en **débordement** $\Rightarrow$
Algorithme de résolution de collision

Définition $HACHAGE (BD)$

- Division d'un fichier en paquet de taille fixe
- A l'intérieur d'un paquet les tuples ne sont pas triés
- La clé est convertie en un numéro de paquet à l'aide d'une fonction de hachage

Méthode

1. Considérer la valeur de la clé comme une suite de bits
2. Appliquer une fonction à valeur dans $[0, N-1]$
(N = nombre de paquets primaires)

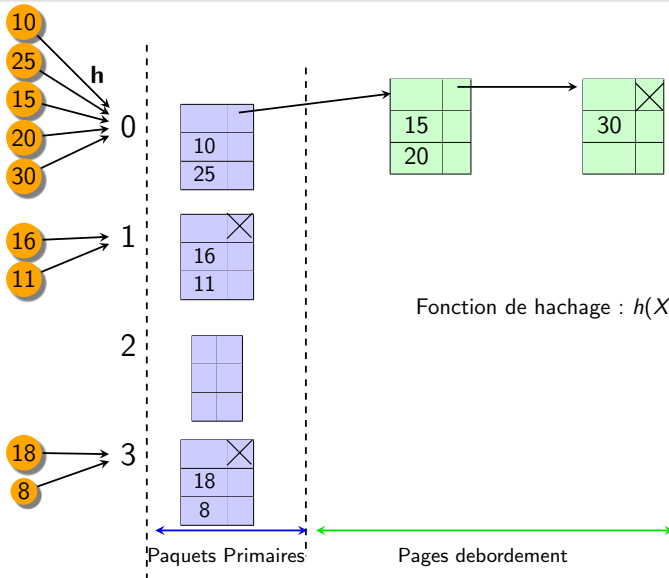
Exemples de fonctions de hachage : Extraction, pliage, division etc.

Caractéristiques d'une bonne fonction de hachage

Elle doit :

- Etre calculable rapidement
- Assurer une répartition uniforme

Hachage statique



Avantages

- En absence de débordement, accès sur clé de placement très performant
- Bien adapté pour des relations de taille moyenne à faible croissance

Inconvénients

- Pas d'accès séquentiel trié
- Pas de question d'intervalle
- Paquets primaires pouvant être vides
- Un paquet peut devenir plein
- Résolutions des collisions par gestion de paquets de débordement :
 - **Chainage séparé**
 - Chainage imbriqué
 - Adressage ouvert
- Dégradation des performances de l'accès direct quand débordement
- Réorganisation périodique
- Mal adapté aux relations à forte croissance

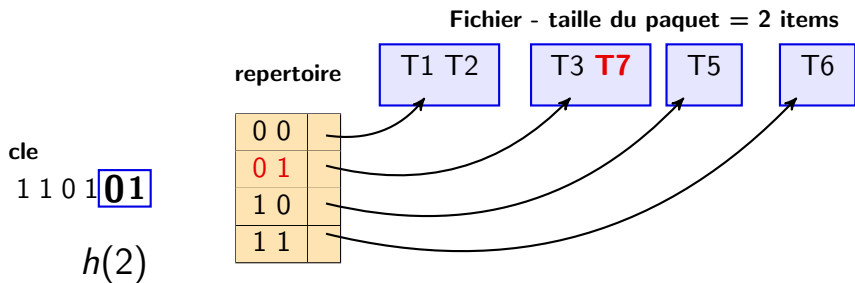
⇒ Organisation par **hachage dynamique**

Définition HACHAGE DYNAMIQUE

- Objectif : adaptation du hachage à la dynamique des fichiers
- Principe : changer de fonction de hachage de manière dynamique lors de la croissance du fichier, en utilisant progressivement la chaîne de bits résultant de l'application de la fonction de hachage.

- Hachage dynamique
- Hachage virtuel
- **Hachage extensible**
- Hachage linéaire

Hachage extensible (Exemple)

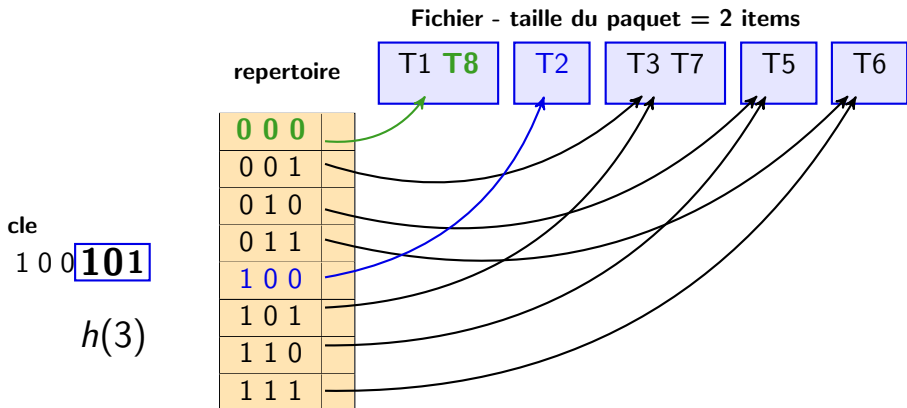


T7 : 1 1 0 1 0 1 ?

T8 : 0 1 0 0 0 0 ?

Exemple adapté du cours de J.Y. Antoine IUT Blois

Hachage extensible (Exemple)



T1 : 0 0 0 0 0 0

T8 : 0 1 0 0 0 0

T2 : 0 0 0 1 0 0

T9 : 1 0 0 1 0 1 ?

Hachage extensible (Exemple)

Fichier - taille du paquet = 2 items

repertoire

0 0 0	
0 0 1	
0 1 0	
0 1 1	
1 0 0	
1 0 1	
1 1 0	
1 1 1	



cle

1 0 0 **101**

$h(3)$

T9 : 1 0 0 1 0 1

T3 : ... 0 0 1

T7 : ... 1 0 1

Définition HACHAGE EXTENSIBLE

Structure :

- Un catalogue dynamique contenant les adresses des paquets
- Des paquets de hachage contenant les données et croissant dynamiquement

Principes :

- Utilisation progressive du hash-code de N bits
- La fonction de hachage adresse un catalogue contenant les adresses des paquets
- Taille du catalogue $= 2^M$
- Extension du fichier quand un paquet devient plein par éclatement de ce dernier

Avantages

- Accès sélectif sur clé de placement toujours performant
- Adapté à des relations à forte dynamicité

Inconvénients

- Croissance exponentielle de la taille du catalogue
- Pas d'accès séquentiel trié
- Pas de question d'intervalle

Conclusion sur l'utilisation du hachage dynamique par rapport aux arbres-B

+ rapide qu'un arbre-B ...

hachage : $O(1)$

B-Tree : $O(\log(n))$

... quand c'est possible de l'utiliser

- les données ne sont pas triées : pose entre autre problème pour répondre à des requêtes qui renvoient les données triées
- les opérateurs disponibles sont limités à '=' et '<>' (arbre-B : '=', '<>', '<=', '>=', 'between')